Analysis of the Portuguese Digital Wallet Mobile Application

Jorge Miguel Amorim Marques

Abstract

This report presents a security analysis of the Portuguese Digital Wallet, an EBSI conformant SSI wallet for Verifiable Credentials and Verifiable Presentations. The assessment combined source-code review, Burp Suite interception of the deployed iOS app, TLS analysis, local backend testing, authentication testing, and privacy review. The main finding is that a trusted-CA MITM setup successfully intercepted sensitive wallet traffic during normal creation and recovery flows. Semi-redacted evidence shows seed phrase fields, private-key-related fields, DID material, and credential data in app-backend communication. A captured credential request was replayed successfully, and the returned credential was decoded locally as a readable JWT-style Verifiable Credential. The report also identifies supporting issues in backend-assisted key handling, local storage, OAuth randomness, session re-authentication, CORS, URL fetching, validation flags, and privacy transparency. The analysis uses only test wallet data and concludes with mitigations for improving key custody, mobile transport hardening, API controls, and wallet privacy documentation.

1. Introduction

The Portuguese Digital Wallet is an EBSI conformant SSI wallet used for Verifiable Credential and Verifiable Presentation workflows. This makes it security-sensitive, since the application handles wallet creation, credential issuance, credential storage, wallet recovery, and credential presentation.

This report presents a preliminary security analysis of the deployed iOS application and its supporting backend API. The assessment follows the course project scope by covering software security, network security, authentication, and privacy under a defined threat model.

The main result is a successful trusted-CA man-in-the-middle attack against the deployed iOS app. During normal wallet creation and recovery flows, the intercepted traffic exposed seed phrase fields, private-key-related fields, DID material, and credential data. The analysis then used source-code review and local backend tests to explain why this behavior occurs and to evaluate its impact.

All dynamic testing used my own test wallet data. No other user data was accessed. Production denial-of-service testing, brute force testing, and broad fuzzing were not performed. More intrusive tests, such as URL-fetching and availability stress tests, were reproduced only against a local backend instance.

2. Scope and Ethical Boundaries

The assessment covered the deployed iOS application, the deployed backend API, the frontend source code, the backend source code, and a local backend instance used for controlled experiments. The dynamic analysis was performed on an iPhone 11 running iOS 17.6.1, using a test wallet created for this project. The main practical tests included Burp Suite interception of the deployed app, TLS configuration analysis, request replay, local credential decoding, local backend recovery tests, CORS testing, SSRF-style URL-fetching tests, and a local availability stress test.

The analysis was limited to my own test wallet and did not access data belonging to other users. Production stress testing, brute force testing, broad fuzzing, and infrastructure attacks were not performed. Tests with higher operational risk, such as URL-fetching and availability experiments, were reproduced only against the local backend. This follows the project requirement to perform the analysis under rigorous ethical standards and avoid degrading deployed services.

3. Threat Model and Security Properties

3.1 Threat Model

The analysis considers five main attacker models.

The first attacker is a network adversary. This attacker controls or observes the network used by the mobile device, for example through a malicious Wi-Fi access point, compromised network equipment, or a configured proxy. The attacker is not assumed to break TLS cryptography directly. Instead, the relevant capability is attempting a trusted-CA man-in-the-middle attack, where the device accepts an attacker-controlled certificate authority. This model was tested in practice using Burp Suite.

The second attacker has access to the user device or local app environment. This includes a lost unlocked phone, a phone left unattended after authentication, a jailbroken device, forensic extraction tools, or malware with elevated access. This attacker is relevant for analyzing local wallet storage, session re-authentication, clipboard use, and the risk of storing DID/private-key-related material outside platform-secure storage.

The third attacker targets the backend API. This attacker sends requests directly to exposed endpoints, replays captured requests, submits malformed inputs, or provides attacker-controlled URLs in protocol parameters. This model is relevant for request replay, permissive CORS behavior, URL-fetching behavior, and availability testing.

The fourth attacker is a malicious or compromised issuer, verifier, or protocol endpoint. This attacker was not fully exploited in the dynamic tests, but it is part of the relevant threat model because the wallet processes QR-based credential offers, issuer metadata, presentation request URIs, verifier identifiers, and external protocol URLs. I tested one related behavior locally: the backend followed a user-controlled credential_offer_uri, confirming that external protocol URLs require strict validation.

The fifth attacker targets availability. This attacker tries to make backend functionality unavailable by repeatedly calling expensive endpoints, such as key generation, recovery, credential issuance, presentation processing, or endpoints that trigger outbound URL fetching. This was not tested against production for ethical reasons. A local stress test was used instead.

3.2 Attack Surface

The attack surface includes the iOS mobile application, the backend API, the app-backend network channel, local wallet storage, credential issuance flows, wallet recovery flows, credential presentation flows, QR-code inputs, issuer and verifier URLs, clipboard operations, PDF export, and session state after authentication.

Credential offer URLs and presentation request URIs were included in the attack surface. The credential-offer URL-fetching behavior was tested locally through the credential_offer_uri parameter. Full malicious verifier testing was left out of scope.

The most security-sensitive flows are wallet creation, wallet import/recovery, DID credential issuance, and credential presentation. These flows handle seed phrases, DID material, private-key-related fields, credential JWTs, and verifier or issuer metadata. The backend API is also an important attack surface because it exposes endpoints for DID generation, recovery, credential issuance, credential offer processing, and presentation-related flows.

3.3 Security Properties

Confidentiality is critical because seed phrases, private-key-related material, credential JWTs, and DID data should not be exposed to network attackers, unnecessary backend components, logs, or local attackers.

Integrity is required because credentials, presentations, DID documents, and protocol messages must not be modified without detection. A wallet that accepts modified or

unvalidated credential material risks breaking trust assumptions in the credential lifecycle.

Authenticity is important because issuers, verifiers, holders, and backend services must be correctly identified. The wallet should only trust valid issuers and verifiers, and credential presentation should prove control of the correct holder material.

Privacy is important because wallet activity reveals information about the holder, credentials, issuers, verifiers, credential types, and presentation contexts. Even when data is necessary for wallet functionality, users should understand what is processed locally, what is sent to the backend, and what is disclosed to issuers or verifiers.

Availability matters because users depend on backend-supported operations such as wallet creation, recovery, credential issuance, and presentation. Endpoints that perform key generation, signing, validation, or outbound URL fetching should resist abuse through rate limiting, timeouts, and monitoring.

Accountability and non-repudiation are also relevant. Credential issuance and presentation flows should provide evidence of what was issued or presented, while avoiding designs where private-key material is unnecessarily exposed to parties that should not control the holder's wallet.

4. Methodology

The assessment combined static analysis, dynamic network testing, local backend experiments, authentication testing, and privacy review. This was done to cover the four project areas: software security, network security, authentication, and privacy.

4.1 Static Source-Code Review

The frontend and backend source code were reviewed to identify security-sensitive logic. Since source code was available, source-code review was prioritized over decompilation because it preserves function names, comments, types, and architecture. The frontend review focused on storage, cryptographic usage, OAuth/PKCE logic, certificate pinning, permissions, and privacy-relevant dependencies. The backend review focused on DID generation, mnemonic recovery, credential issuance, signing logic, CORS, URL fetching, validation flags, and API hardening.

The main evidence from this phase consisted of code screenshots and file paths showing where sensitive operations occur. These included AsyncStorage usage, backend DID generation, backend recovery from mnemonic, server-side signing with private-key-related material, Math.random usage in OAuth-related code, and disabled EBSI validation flags.

4.2 Dynamic Network Testing

Burp Suite Community was used as a trusted-CA MITM proxy. The iPhone was connected to the Mac through USB internet sharing, and the app traffic was routed through Burp using Potatso. The Burp CA certificate was installed and trusted on the iPhone. This setup allowed inspection of HTTPS traffic generated by the deployed iOS app.

The network tests covered wallet creation, wallet import/recovery, DID credential issuance, and request replay. Sensitive values in report figures are semi-redacted: field names, endpoints, request/response direction, and status codes are kept visible, while full seed phrases, private keys, DID values, and credential tokens are partially hidden.

TLS server configuration was assessed separately using Qualys SSL Labs against api.digitalwallet.pt. This was used to distinguish server-side TLS configuration from app-level MITM resistance.

4.3 Local Backend Proof-of-Concept Tests

A local instance of the backend API was used for tests that should not be performed against production. These included recovery validation,

DID credential issuance, CORS checks, credential_offer_uri URL-fetching behavior, and local availability stress testing.

The recovery test generated a DID locally, used the returned mnemonic to recover it through the backend, and confirmed that the same seed, private key, and DID were reconstructed. The URL-fetching test used a local HTTP server to confirm that the backend follows a credential_offer_uri supplied through the credential-offer flow. The availability test used autocannon against localhost only, to observe behavior under repeated local requests without affecting the deployed service.

4.4 Authentication and Privacy Testing

Authentication testing was performed manually on the deployed iOS app. The tests checked Face ID behavior after full app restart, backgrounding, phone lock/unlock, wallet deletion, and credential deletion. Privacy testing covered observed iOS permissions, App Store privacy labels, the linked privacy policy, tracker/dependency review, and privacy-sensitive wallet data flows.

4.5 Limitations

The assessment did not include access to other users' data, production stress testing, brute force testing, broad fuzzing, full infrastructure review, or direct extraction of the production iOS app storage container. Some findings are therefore classified as confirmed, while others are treated as conditional or hardening observations. For example, disabled EBSI validation flags and credential_offer_uri fetching were confirmed in code or local tests, but their full production impact depends on deployment configuration and downstream validation.

5. Network Security Analysis

5.1 TLS Configuration

Before testing interception, I assessed the TLS configuration of api.digitalwallet.pt using Qualys SSL Labs. The server received an overall grade of A. The report showed support for TLS 1.3 and TLS 1.2, while older protocols such as TLS 1.1, TLS 1.0, SSL 3, and SSL 2 were disabled. No major TLS vulnerabilities were reported.

This result suggests that the main transport-layer weakness was not the backend TLS configuration itself. Instead, the relevant weakness was at the mobile app trust layer: after the Burp CA certificate was installed and trusted on the iPhone, the app accepted the intercepted HTTPS connection and did not block the MITM attempt.

5.2 MITM Setup

The MITM test was performed using Burp Suite Community. The iPhone was connected to the Mac through USB internet sharing, and Potatso was used to route iPhone traffic through Burp at 192.168.2.1:8080. The Burp CA certificate was installed and fully trusted on the iPhone. The test device was an iPhone 11 running iOS 17.6.1.

This setup allowed Burp to intercept HTTPS traffic generated by the deployed iOS app. Only my own test wallet was used. Sensitive values are semi-redacted in the figures.

5.3 Wallet Creation Flow

During wallet creation, the app contacted api.digitalwallet.pt. Two important endpoints were observed:

GET /holder/get_new_did

POST /holder/get_did_credential

In the GET /holder/get_new_did response, the backend returned DID material, including seed and private-key-related fields. The seed value matched the seed phrase displayed in the app. In the following POST /holder/get_did_credential request, the app sent DID/private-key-related material back to the backend. The response contained a credential field.

This shows that sensitive wallet material was visible in the intercepted app-backend communication during normal wallet creation. The most sensitive exposure was the seed/private-key-related material, because it is

linked to wallet control. The credential value is also privacy-sensitive, because it contains Verifiable Credential data.

Figure 1: POST /holder/get_did_credential request/response showing privateKey and seed fields

Figure 2: POST /holder/get_did_credential request/response showing credential field

5.4 Wallet Import and Recovery Flow

The wallet import flow confirmed the same issue from another angle. When importing the test wallet, the app sent the mnemonic recovery phrase to:

POST /holder/recover_did_from_seed

The backend response returned DID and private-key-related material. Immediately afterwards, the app called:

POST /holder/get_did_credential

again sending DID/private-key-related material and receiving a credential response.

This confirms that wallet recovery is server-assisted. The recovery phrase is not handled only locally on the device. Instead, it is transmitted to the backend, where the wallet material is reconstructed and returned.

Figure 3: POST /holder/recover_did_from_seed response showing seed/privateKey fields

5.5 Replay of Captured Credential Request

To test practical impact, I sent the captured POST /holder/get_did_credential request to Burp Repeater without modifying it. The backend accepted the replayed request and returned HTTP/2 201 Created with a credential field in the response.

This shows that the captured request contained enough information to repeat a sensitive credential operation for the test wallet. This does not prove access to other users' data. It shows that an attacker who captures this request, for example through a successful MITM position, proxy access, device compromise, or logs, may replay it and obtain credential material again for that wallet.

5.6 Credential Decoding

The returned credential was copied to a local file and decoded with a local Python script. No online decoder was used. The credential had three JWT parts. The decoded header contained fields such as alg, kid, and typ. The payload contained fields such as iat, exp, jti, sub, iss, nbf, and vc. The vc object contained Verifiable Credential fields, including issuer, credentialSubject, validity dates, and credentialSchema. The script is included in a supplementary code file submitted with the report.

This confirms that the credential is a signed JWT-style Verifiable Credential. The signature protects integrity, but the payload is readable after base64url decoding. Therefore, interception affects confidentiality and privacy,

even if the credential cannot be modified without invalidating the signature.

Figure 4: local JWT decoding output showing header keys, payload keys, vc keys, and credentialSubject keys

5.7 Network Security Conclusion

The deployed backend TLS configuration appears strong based on SSL Labs. The main network security issue is therefore not weak TLS. The issue is that the deployed iOS app accepted the trusted Burp CA certificate and allowed the MITM proxy to inspect sensitive wallet traffic. During normal wallet creation and recovery flows, seed phrase fields, private-key-related fields, DID material, and credential data were visible. The successful replay and credential decoding tests show that the impact goes beyond passive observation.

6. Software Security Analysis

6.1 Backend-Assisted Key Lifecycle

The source-code review showed that the wallet uses a backend-assisted key lifecycle. In the backend, the /holder/get_new_did endpoint generates wallet DID material and returns fields such as seed, privateKey, x, and y. This matches the dynamic Burp evidence from the deployed app, where the same type of material appeared during wallet creation.

The recovery flow follows the same model. The /holder/recover_did_from_seed endpoint receives a mnemonic recovery phrase, validates it, derives the corresponding key material, and returns the DID object with seed and private-key-related fields. A local backend validation test confirmed that using the generated mnemonic in the recovery endpoint reconstructed the same seed, private key, and DID.

The credential issuance flow also uses backend-side signing. The /holder/get_did_credential endpoint receives a DID object from the app, including private-key-related material, and uses it to create a Verifiable Credential JWT. This was confirmed in three ways: backend source code, Burp traffic from the deployed app, and a local backend test that returned a jwt_vc credential.

This design does not look like a small implementation mistake. It appears to be an architectural choice where the backend assists with DID generation, recovery, and credential-related signing. This may simplify EBSI conformance and development, but it increases trust in the backend. In a stronger holder-controlled wallet design, mnemonic handling, key derivation, and signing should remain on the user device, and the backend should not receive seed phrases or private-key material.

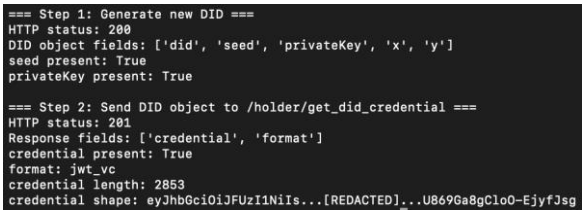

Figure 5: Backend /holder/get_new_did code showing returned DID, seed, privateKey, x, and y fields.

6.2 Local Storage of Wallet Material

The frontend source code stores the DID object using React Native AsyncStorage. The saveDID function serializes the DID object and writes it with AsyncStorage.setItem(). The code comment describes the DID object as containing identity information, the DID string, the private key, and public key coordinates. The frontend static review also identified AsyncStorage storage of DID/private-key material and credential JWTs.

This was not directly extracted from the production iOS app storage container. The test device was not jailbroken, and iOS sandboxing prevents direct access to another app's container in normal conditions. Therefore, this finding is reported as a source-code-supported local storage design weakness, not as a demonstrated production storage extraction attack.

The security concern is that AsyncStorage is general-purpose app storage, not platform-secure storage for cryptographic secrets. A

random remote attacker does not read AsyncStorage directly. The risk becomes relevant if an attacker has device-level access, a jailbroken device, forensic tooling, malware with elevated access, or access to a compromised local app environment. For an EBSI conformant wallet, DID private-key material should be protected using stronger platform mechanisms such as iOS Keychain, Secure Enclave-backed keys, or an equivalent secure storage design.

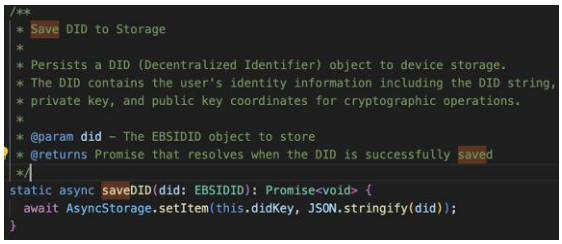

Figure 6: Frontend saveDID function showing AsyncStorage

6.3 Weak Randomness in OAuth/PKCE Logic

The frontend code uses Math.random() in two security-sensitive places in helpers/ebsi.ts. First, generateCodeVerifier() creates the PKCE code verifier by selecting characters using Math.random(). Second, getAuthorizationParameters() uses Math.random().toString(36).substring(2, 10) as a fallback state value when issuer_state is not provided.

PKCE and OAuth state values protect authorization flows. The PKCE verifier acts as a temporary secret during the authorization-code exchange, while the state parameter supports request correlation and CSRF protection. Math.random() is not designed as a cryptographically secure random source, and the fallback state value is also short.

This finding does not directly expose wallet keys or credentials by itself. Its impact is lower than the MITM and key lifecycle findings. However, it weakens protocol-level protection around credential issuance and authentication flows. A safer implementation should use a cryptographically secure source such as expo-crypto random bytes, followed by base64url encoding.

Figure 7: generateCodeVerifier() showing Math.random()

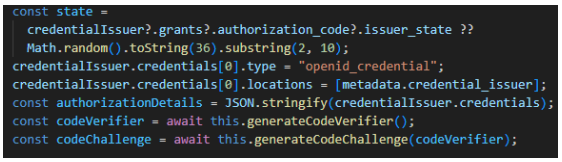

Figure 8: fallback state generation showing Math.random().toString(36)

6.4 Disabled EBSI Validation Flags

The backend source code contains selected flows where EBSI-related validation is explicitly disabled. In /holder/get_did_credential, the backend calls createVerifiableCredentialJwt with skipValidation, skipAccreditationsValidation, skipCredentialSubjectValidation, and skipStatusValidation set to true. A similar set of skip flags appears in /verifier/issue_presentation when creating a Verifiable Presentation. The disabled-validation review confirmed these two code paths.

This should be treated as a conditional finding. The code clearly disables local validation checks, but the assessment did not prove that invalid credentials are accepted in production. Some validation may still happen through external EBSI services, issuers, or downstream verifiers. The impact also depends on whether these paths are intended only for conformance testing or are reachable in production flows.

Even with that limitation, the flags deserve attention. If used outside an explicit test or conformance mode, they weaken assurance around issuer accreditation, credential subject binding, general credential validation, and status or revocation checking. A safer design would enable validation by default and only allow skip flags in a clearly separated test mode.

Figure 9: /holder/get_did_credential showing skipValidation flags

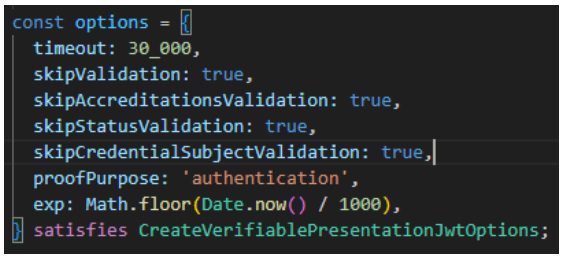

Figure 10: /verifier/issue_presentation showing skipValidation flags

The software findings explain why sensitive wallet material appeared in network traffic. The next section analyzes how the app protects access to the wallet after authentication.

7. Authentication Analysis

7.1 Local Authentication

The app uses Face ID as a local authentication gate before allowing access to the wallet. In testing, after fully closing and reopening the app, the app returned to the “Autenticar” screen and required Face ID before allowing access again. This is a positive control because it prevents immediate access to the wallet after a fresh app launch.

However, Face ID protects access to the app interface. It does not, by itself, protect private-key material at rest or prevent backend-assisted recovery and signing flows. This distinction matters because the earlier findings showed that sensitive wallet material is transmitted and handled by the backend during creation, recovery, and credential issuance.

7.2 Background and Re-authentication Behavior

The app did not require Face ID again when returning from the background. I tested this by opening the wallet, switching to another app for 10 minutes, locking the phone for 2 minutes, unlocking the phone, and returning to the wallet app. The app opened directly back into the wallet without a new Face ID prompt.

This creates a local access risk. If a user authenticates once and then leaves the phone unlocked, or if another person gains access shortly after the device is unlocked, the wallet may remain accessible without fresh biometric authentication. This is less severe than seed or private-key exposure, but it is relevant for a wallet that stores and presents credentials.

A stronger session policy would re-lock the wallet after backgrounding, after device lock/unlock, or after a short inactivity timeout. A practical design could require Face ID again after 1 to 5 minutes in the background, or immediately after the device is locked.

7.3 Wallet Recovery Authentication

The wallet recovery flow uses a mnemonic recovery phrase. In the deployed app, importing a wallet sent the mnemonic to the backend endpoint /holder/recover_did_from_seed. The backend response returned DID and private-key-related material. This was already shown in the network analysis, but it also affects authentication: the mnemonic acts as the recovery secret for wallet control.

A local backend test confirmed that the same mnemonic reconstructed the same seed, private key, and DID. This means that whoever obtains the mnemonic can recover the wallet identity. For this reason, the mnemonic should be treated as equivalent to the wallet private key.

The main concern is not that mnemonic recovery exists. Recovery phrases are common in wallet systems. The concern is that recovery is server-assisted, so the mnemonic is

transmitted to the backend instead of being processed only on the user device.

7.4 Wallet and Credential Deletion

The app provides two destructive actions: “Delete all credentials” and “Delete wallet”. Both actions showed confirmation dialogs before deletion. This is positive because the app asks for confirmation before removing wallet data.

When “Delete wallet” was confirmed, the app returned to the “Authenticate” screen. After pressing “Authenticate” again, Face ID was required, and the app showed the wallet setup options “Import wallet” and “Create”. Burp did not show any backend request during this action, suggesting that wallet deletion is local-only.

When “Delete all credentials” was confirmed, the credential disappeared, the app stayed inside the wallet, and Burp did not show any backend request. This suggests that credential deletion is also local-only and separate from wallet deletion.

The main improvement is user transparency. The deletion dialogs ask for confirmation, but they do not explain what data is deleted, whether deletion is local-only, or whether recovery requires the seed phrase. For a wallet application, destructive actions should clearly explain their security and recovery consequences.

7.5 Authentication Summary

The app has a useful local authentication gate through Face ID, and deletion actions require confirmation. However, the wallet does not appear to re-lock after backgrounding or phone lock/unlock, and wallet recovery depends on sending the mnemonic to the backend. The recommended improvements are to add re-authentication on resume or after timeout, clarify deletion consequences, and redesign recovery so mnemonic processing happens locally where possible.

8. Privacy Analysis

8.1 Trackers and Analytics

The reviewed source code did not show confirmed analytics, advertising, crash-reporting, social-login, or tracker SDK behavior. The privacy/tracker review found no confirmed Firebase, Google Analytics, Sentry, Crashlytics, Mixpanel, Segment, Amplitude, advertising SDKs, IDFA/ADID usage, Facebook/Meta SDK, or social-login tracking behavior in the reviewed frontend and backend source code.

This is a positive result. It suggests that the app does not appear privacy-invasive in the traditional tracker or advertising sense. Exodus was not used because the selected target was the iOS application, while Exodus is mainly useful for Android APK tracker analysis. Instead, privacy analysis was based on source-code dependency review, App Store privacy labels, iOS permission checks, privacy policy review, and observed network traffic.

8.2 Permissions

The observed iOS permissions were Face ID, Camera, and Siri & Search. Face ID is justified because the app uses local authentication before wallet access. Camera access is justified because the app scans QR codes for credential issuance and presentation flows. No Location, Microphone, Contacts, Photos, Bluetooth, or Notifications permission was observed during the iOS test.

8.3 Sensitive Wallet Data Processing

The main privacy concern is not third-party tracking. The main concern is that the app processes sensitive wallet data as part of core EBSI flows. This includes DID/private-key-related material, seed phrases, credential JWTs, QR payloads, issuer/verifier metadata, and presentation-related data.

This was confirmed by multiple tests. The MITM analysis showed sensitive wallet material in app-backend traffic during creation and recovery flows. The local credential decoding

test showed that the returned credential was a signed JWT-style Verifiable Credential with readable payload structure. The source review also confirmed local storage of DID/private-key-related material and credentials through AsyncStorage.

The privacy implication is that users should be clearly informed about what data is stored locally, what data is sent to the backend, and what data may be disclosed to issuers, verifiers, authorization servers, or EBSI services during wallet flows.

8.4 Clipboard and Sharing

The privacy review found that the app can copy the seed phrase and DID to the clipboard. This is privacy-sensitive because the seed phrase is equivalent to the wallet recovery secret. Clipboard content may be exposed to the operating system or to other apps. The app should warn users before copying a seed phrase and should consider avoiding seed phrase clipboard copying entirely.

The app also supports PDF export and OS sharing of credentials. It may disclose credential attributes to external apps.

8.5 Privacy Policy and Transparency

The App Store privacy label states that the developer does not collect data from the app. This aligns with the source review, where no tracker SDKs were confirmed. However, the linked privacy policy points to a general TecMinho privacy policy rather than a wallet-specific privacy notice.

For an EBSI conformant wallet, a dedicated privacy notice would improve transparency. It should explain what wallet data is processed locally, what data is sent to the backend, whether seed or private-key-related material is processed by the backend, which issuers/verifiers receive data, what logs may contain, what “delete credentials” and “delete wallet” remove, and how users can exercise GDPR rights for wallet-related data.

8.6 Privacy Summary

The app shows good privacy behavior in the traditional tracker sense: no confirmed analytics, advertising, crash reporting, social-login, or tracker SDKs were found, and iOS permissions were minimal. The main privacy risk comes from wallet-specific data handling. Sensitive wallet material and credential data are processed during normal EBSI flows, so the app should improve transparency, reduce exposure of seed/private-key-related material, avoid clipboard exposure for recovery secrets, and provide a wallet-specific privacy notice.

9. Backend Hardening and Availability

9.1 Open CORS and Unauthenticated Sensitive Endpoints

The backend locally returned permissive CORS headers. A request to /holder/get_new_did with the header Origin: https://evil.example returned HTTP 200 OK and Access-Control-Allow-Origin: *. The response also contained wallet key material fields, including did, seed, privateKey, x, and y.

A second local preflight test against /holder/recover_did_from_seed returned HTTP 204 No Content with Access-Control-Allow-Origin: *, Access-Control-Allow-Methods: GET,HEAD,PUT,PATCH,POST,DELETE, and Access-Control-Allow-Headers: content-type.

This confirms that, in the local backend configuration, arbitrary browser origins are allowed to call the API. This does not prove that another user’s wallet material is exposed, because /holder/get_new_did generates new key material rather than returning an existing user’s wallet. Still, for an EBSI conformant wallet backend, permissive CORS combined with unauthenticated endpoints that handle sensitive wallet operations is weak hardening.

A safer configuration would restrict CORS to trusted frontend origins, limit allowed methods and headers, and avoid exposing sensitive

endpoints without authentication or abuse protection.

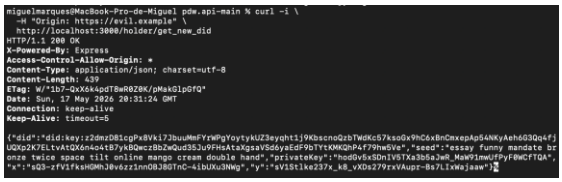

Figure 11: Local CORS GET test showing Access-Control-Allow-Origin: * and returned field names

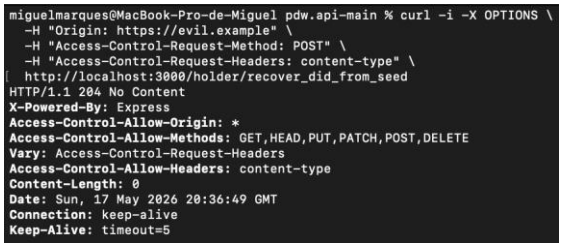

Figure 12: Local CORS OPTIONS preflight test showing allowed methods and headers

9.2 User-Controlled credential_offer_uri Fetching

The backend also follows URLs supplied through the credential-offer flow. In a local test, I supplied a credential_offer_uri pointing to a local HTTP server. The backend requested /test-offer from that server, and after a valid JSON file was created, the backend fetched and returned the test JSON.

This confirms that the backend performs outbound requests to locations controlled by credential-offer input. This behavior is useful for issuer discovery, but it creates an SSRF-capable pattern if deployed without restrictions. A malicious credential offer could try to make the backend request internal services, localhost resources, cloud metadata endpoints, or other private network targets.

The test was performed only locally. No internal systems or production infrastructure were targeted. The production impact depends on deployment protections such as URL allowlists, private-network blocking, egress filtering, WAF rules, and reverse proxy controls.

Recommended mitigations include requiring HTTPS, allowlisting trusted issuer domains, blocking localhost and private IP ranges, blocking cloud metadata IPs, setting strict timeouts, limiting response size, and returning controlled 4xx errors for invalid credential offers.

Figure 13: Local Python server showing GET /test-offer

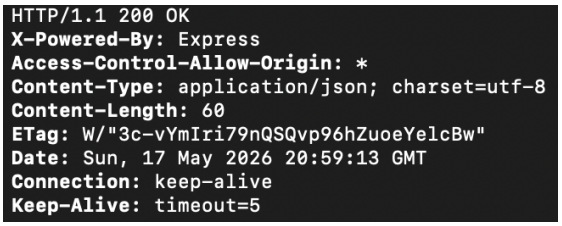

Figure 14: Backend response showing returned test JSON

9.3 Local Availability Stress Test

Availability was considered because several backend endpoints perform relatively expensive operations, such as DID generation, recovery, credential issuance, signing, presentation processing, and outbound URL fetching. Production denial-of-service testing was not performed for ethical reasons. Instead, I ran a local stress test against the backend instance using autocannon.

The tested endpoint was /holder/get_new_did. This endpoint generates a DID, seed phrase, private-key-related material, and public key coordinates. In the stronger local test, autocannon used 100 concurrent connections for 15 seconds. The backend handled around 3,000 requests, with an average throughput of about 172 requests per second. Average latency increased to about 456 ms, the maximum latency reached about 9.7 seconds, and 20 timeouts occurred. No explicit rate-limiting response such as HTTP 429 Too Many Requests was observed.

This does not prove that the production deployment is vulnerable to denial of service, because production infrastructure may include rate limiting, load balancing, WAF rules, or cloud DoS protection outside the reviewed source code. The local result still shows that the

backend application itself accepted repeated unauthenticated requests to a key-generation endpoint without visible throttling.

Recommended mitigations include rate limiting, request throttling, monitoring, abuse detection, and authentication or client trust controls before expensive operations where possible.

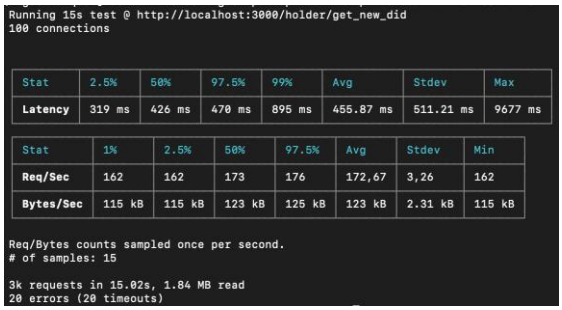

Figure 15: autocannon local stress test output

9.4 Backend Hardening Summary

The local backend tests identified several hardening gaps: permissive CORS, unauthenticated access to sensitive wallet endpoints, user-controlled outbound URL fetching, and no visible application-level throttling during local stress testing. These findings should be treated as backend hardening issues rather than confirmed production compromise. Their final impact depends on deployment controls that were not visible in the provided source code.

10. Recommendations

10.1 Keep Seed Phrases and Private Keys on the Device

The highest-priority recommendation is to remove seed phrases and private-key-related material from backend flows. Wallet creation, recovery, and signing should happen on the user device where possible. The backend should not generate mnemonic seed phrases, receive recovery phrases, or sign using private-key material sent by the app.

A stronger design would generate and store holder key material locally, then send only public identifiers, signed proofs, signed presentations, or signed requests to the backend. This would reduce the impact of MITM, backend compromise, backend logging mistakes, and request replay.

10.2 Use Platform-Secure Storage

The frontend should avoid storing DID/private-key-related material in AsyncStorage. AsyncStorage is suitable for general app state, but not for wallet secrets. Private-key-related material should be protected using iOS Keychain, Secure Enclave-backed keys, or an equivalent secure storage design. Credentials and temporary protocol state should also be minimized, encrypted where appropriate, and cleared after use.

10.3 Strengthen Mobile Transport Security

The backend TLS configuration appeared strong, but the deployed iOS app accepted the trusted Burp CA certificate and allowed sensitive traffic to be inspected. For an EBSI conformant wallet, the team should consider certificate pinning or another mobile trust-hardening approach. This would not replace TLS, but would make trusted-CA MITM attacks harder.

This should be implemented carefully, because certificate pinning can break availability if certificates are rotated incorrectly. A safer approach is public-key pinning with backup pins and a clear rotation process.

10.4 Remove Sensitive Material from Requests

The /holder/get_did_credential flow should not require the app to send private-key-related material to the backend. If the backend needs proof of holder control, the app should create the proof locally and send only the signed result. This would also reduce replay impact, because a captured request would not contain reusable wallet secrets.

10.5 Fix OAuth Randomness

The PKCE code verifier and fallback state value should be generated with a cryptographically secure random source. In an Expo React Native app, expo-crypto can provide secure random bytes. The state value should also be long enough to resist guessing and should be validated during the flow.

10.6 Enable Validation Outside Test Mode

The EBSI validation skip flags should not be active in production flows unless there is a documented reason and another trusted layer performs equivalent validation. Validation should be enabled by default for credential status, issuer accreditation, credential subject binding, schema trust, expiration, and general credential/presentation validity. If skip flags are needed for conformance testing, they should be isolated behind an explicit test mode.

10.7 Harden Backend API Controls

The backend should restrict CORS to trusted origins instead of allowing all origins. Sensitive endpoints should also have rate limiting, request throttling, monitoring, and abuse detection. Expensive operations such as key generation, recovery, credential issuance, signing, and outbound URL fetching should have stronger controls.

10.8 Restrict External URL Fetching

The credential_offer_uri fetching behavior should be restricted. The backend should require HTTPS, allowlist trusted issuer domains where possible, block localhost and private IP ranges, block cloud metadata endpoints, add timeouts, limit response size, and return controlled 4xx errors for invalid offers.

10.9 Improve Session Re-locking

The app should re-lock the wallet after backgrounding, after device lock/unlock, or after a short inactivity timeout. A practical policy would require Face ID again after 1 to 5 minutes in the background, or immediately after the phone is locked.

10.10 Improve Privacy Transparency

The App Store privacy label and lack of tracker SDKs are positive. However, the wallet should have an app-specific privacy notice. It should explain what data is stored locally, what is sent to the backend, what issuers and verifiers receive, whether seed/private-key-related material is processed by the backend, what logs may contain, and what “delete wallet” and “delete credentials” remove.

10.11 Improve User Warnings for Sensitive Actions

The app should improve warnings around clipboard copying, wallet deletion, credential deletion, and PDF sharing. Users should understand that a seed phrase can recover the wallet, that copied clipboard content may be exposed outside the app, that exported PDFs may contain credential attributes, and that deletion appears to affect local wallet data only.

11. Conclusion

This project analyzed the Portuguese Digital Wallet as an EBSI conformant SSI wallet using source-code review, real-device MITM testing, local backend experiments, authentication testing, and privacy review. The main result was a successful trusted-CA MITM attack against the deployed iOS app. During normal wallet creation and recovery flows, the intercepted traffic exposed seed phrase fields, private-key-related fields, DID material, and credential data.

The strongest conclusion is that the wallet currently relies on a backend-assisted key lifecycle. The backend participates in DID generation, mnemonic recovery, and credential-related signing operations. This explains why sensitive wallet material appeared in app-backend traffic and why a captured credential request could be replayed successfully. The issue is therefore not only transport interception. It is also an architectural trust issue: the backend handles material that would normally be expected to remain under holder control in a stronger SSI wallet design.

Other findings support the same conclusion. The frontend stores DID/private-key-related material using AsyncStorage, OAuth-related randomness uses Math.random(), selected EBSI validation checks are disabled in backend flows, and local backend tests showed permissive CORS, user-controlled URL fetching, and no visible rate limiting under local stress testing. Authentication and privacy testing showed positive controls such as Face ID and minimal iOS permissions, but also showed missing re-locking after backgrounding and limited wallet-specific privacy transparency.

The most important mitigations are to move mnemonic handling, key derivation, and signing to the device, protect secrets with platform-secure storage, avoid transmitting private-key-related material to the backend, consider certificate pinning or equivalent trust hardening, restrict backend API behavior, and improve privacy documentation. These changes would reduce the impact of MITM, backend compromise, replay, local device compromise, and accidental logging of wallet material.

12. References

PortSwigger, “Configuring an iOS device to work with Burp Suite.” PortSwigger Web Security. https://portswigger.net/burp/documentation/desktop/mobile/config-ios-device

PortSwigger, “Installing Burp’s CA certificate.” PortSwigger Web Security. https://portswigger.net/burp/documentation/desktop/external-browser-config/certificate

Qualys SSL Labs, “SSL Server Test.” https://www.ssllabs.com/ssltest/

OWASP, “MASVS-NETWORK-2: The app performs identity pinning for all remote endpoints under the developer’s control.” OWASP Mobile Application Security. https://mas.owasp.org/MASVS/controls/MASVS-NETWORK-2/

Apple, “Keychain services.” Apple Developer Documentation. https://developer.apple.com/documentation/security/keychain-services

React Native, “Security.” React Native Documentation. https://reactnative.dev/docs/security

React Native AsyncStorage, “React Native Async Storage.” GitHub. https://github.com/react-native-async-storage/async-storage

D. Sakimura, J. Bradley, and N. Agarwal, “Proof Key for Code Exchange by OAuth Public Clients,” RFC 7636, Internet Engineering Task Force, Sep. 2015. https://datatracker.ietf.org/doc/html/rfc7636

Expo, “Crypto.” Expo Documentation. https://docs.expo.dev/versions/latest/sdk/crypto/

W3C, “Verifiable Credentials Data Model v2.0.” W3C Recommendation, May 15, 2025. https://www.w3.org/TR/vc-data-model-2.0/

EBSI, “Request and present Verifiable Credentials.” EBSI Hub. https://hub.ebsi.eu/conformance/build-solutions/holder-wallet-functional-flows

OpenID Foundation, “OpenID for Verifiable Credential Issuance.” https://openid.net/specs/openid-4-verifiable-credential-issuance-1_0.html

MDN Web Docs, “Cross-Origin Resource Sharing (CORS).” Mozilla. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS

OWASP, “Server-Side Request Forgery Prevention Cheat Sheet.” OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html

OWASP, “API4:2023 Unrestricted Resource Consumption.” OWASP API Security Top 10. https://owasp.org/API-

Security/editions/2023/en/0xa4-unrestricted-resource-consumption/

TecMinho, “Política de Privacidade.” https://www.tecminho.uminho.pt/politica-de-privacidade